AE700 Detailed Study Material
Lectures 13 to 17

Prepared from course lectures: Guidance and Control of Unmanned Autonomous Vehicles

March 31, 2026

Contents

1 Lecture 13: Reduced-Order Guidance Models and Straight-Line Path Following 3

1.1 Why reduced-order guidance models are needed . . . 3
1.2 Autopilot-hold loop abstractions used in guidance . . . 3
1.3 Kinematic model for controlled flight . . . 3
1.4 Coordinated-turn relation . . . 3
1.5 Accelerated climb and load factor . . . 4
1.6 Kinematic guidance model options . . . 4
1.7 Dynamic point-mass guidance model . . . 5
1.8 Trajectory tracking vs path following . . . 5
1.9 Straight-line path geometry . . . 5
1.10 Longitudinal straight-line tracking . . . 5
1.11 Lateral straight-line vector-field strategy . . . 6

2 Lecture 14: Orbit Following, Carrot-Chasing, and NLGL 7

2.1 Lyapunov proof for straight-line lateral guidance . . . 7
2.2 Orbit path representation and kinematics . . . 7
2.3 Orbit vector-field command and stability . . . 7
2.4 Practical angle wrapping issue . . . 7
2.5 Classification of path-following methods . . . 8
2.6 Carrot-chasing algorithm . . . 8
2.7 Nonlinear Guidance Law (NLGL) / L1 Logic . . . 8
2.8 Look-ahead distance selection insight . . . 9
2.9 PD/PID vs NLGL in wind . . . 9

3 Lecture 15: Circular NLGL Analysis, Path Manager, Fillets, Dubins Basics 10

3.1 Circular path with nonlinear guidance assumptions . . . 10

3.2 Dynamics of Eta and Beta . . . 10
3.3 Linearization near circular equilibrium . . . 10
3.4 Waypoint path manager motivation . . . 10
3.5 Half-plane switching . . . 10
3.6 Fillet maneuver for smooth line transitions . . . 11
3.7 Path-length correction with fillets . . . 11
3.8 Dubins vehicle and path decomposition . . . 11
3.9 Circle centers and angle operator . . . 12
3.10 Length formulas introduced in Lecture 15 . . . 12

4 Lecture 16: Dubins Completion and UAV Path Planning Algorithms 13
4.1 Remaining Dubins cases . . . 13
4.2 Path planning classes . . . 13
4.3 Point-to-point planning with Voronoi graph . . . 13
4.4 Edge cost with safety-distance penalty . . . 13
4.5 Dijkstra search procedure . . . 14
4.6 RRT for point-to-point planning . . . 14
4.7 Coverage path planning concept . . . 14

5 Lecture 17: Vision-Guided Navigation, Geolocation, and Time-to-Collision 15
5.1 Role of onboard vision . . . 15
5.2 Frames and transformation chain . . . 15
5.3 Camera projection model . . . 15
5.4 Gimbal pointing control . . . 16
5.5 Geolocation setup . . . 16
5.6 Flat-earth geolocation estimate . . . 16
5.7 EKF formulation for geolocation . . . 16
5.8 Target pixel filtering and velocity estimation . . . 17
5.9 Removing apparent motion due to platform/gimbal rotation . . . 17
5.10 Time-to-collision (TTC) . . . 17

6 Consolidated Revision Checklist (Lectures 13–17) 18

1 Lecture 13: Reduced-Order Guidance Models and Straight-Line Path Following

1.1 Why reduced-order guidance models are needed

The full 6-DOF rigid-body equations with aerodynamic force and moment models are too complex for real-time outer-loop guidance design. Hence, reduced-order models are used to:

- preserve dominant closed-loop behavior of the autopilot-vehicle system,
- simplify path-following control design,
- avoid repeated use of full aerodynamic force/moment balance equations.

1.2 Autopilot-hold loop abstractions used in guidance

Common low-order closed-loop models:

$$\dot{V}_a = b_{V_a}(V_{a,c} - V_a), \qquad \dot{\phi} = b_\phi(\phi_c - \phi),$$

$$\ddot{h} = b_{\dot{h}}(\dot{h}_c - \dot{h}) + b_h(h_c - h), \qquad \ddot{\chi} = b_{\dot{\chi}}(\dot{\chi}_c - \dot{\chi}) + b_\chi(\chi_c - \chi),$$

$$\dot{\gamma} = b_\gamma(\gamma_c - \gamma), \qquad \dot{n}_{lf} = b_{n_{lf}}(n_{lf,c} - n_{lf}).$$

All gains are positive and depend on inner-loop implementation.

1.3 Kinematic model for controlled flight

Ground-velocity form:

$$\dot{p}_n = V_g \cos\chi \cos\gamma, \quad \dot{p}_e = V_g \sin\chi \cos\gamma, \quad \dot{h} = V_g \sin\gamma.$$

With airspeed-heading-flight-path variables and wind:

$$\begin{bmatrix} \dot{p}_n \\ \dot{p}_e \\ \dot{h} \end{bmatrix} = V_a \begin{bmatrix} \cos\psi \cos\gamma_a \\ \sin\psi \cos\gamma_a \\ \sin\gamma_a \end{bmatrix} + \begin{bmatrix} w_n \\ w_e \\ -w_d \end{bmatrix}.$$

For constant altitude and negligible vertical wind:

$$\dot{p}_n = V_a \cos\psi + w_n, \qquad \dot{p}_e = V_a \sin\psi + w_e, \qquad \dot{h} = 0.$$

1.4 Coordinated-turn relation

For coordinated turning (no lateral body-frame acceleration):

$$\dot{\chi} = \frac{g}{V_g} \tan\phi \cos(\chi - \psi).$$

Using wind-triangle differentiation and constant-airspeed assumption:

$$\dot{\psi} = \frac{g}{V_a} \tan\phi.$$

This highlights a key idea: turning is achieved through bank angle.

1.5 Accelerated climb and load factor

Force balance normal to velocity direction (with bank):

$$F_{lift}\cos\phi = mV_g\dot{\gamma} + mg\cos\gamma.$$

Define load factor

$$n_{lf} \triangleq \frac{F_{lift}}{mg},$$

then

$$\dot{\gamma} = \frac{g}{V_g}(n_{lf}\cos\phi - \cos\gamma).$$

For constant-climb condition ($\dot{\gamma} = 0$):

$$n_{lf} = \frac{\cos\gamma}{\cos\phi}.$$

1.6 Kinematic guidance model options

Option 1: course and altitude loops

$$\dot{p}_n = V_a\cos\psi + w_n, \quad \dot{p}_e = V_a\sin\psi + w_e,$$
$$\ddot{\chi} = b_{\dot{\chi}}(\dot{\chi}_c - \dot{\chi}) + b_{\chi}(\chi_c - \chi), \quad \ddot{h} = b_{\dot{h}}(\dot{h}_c - \dot{h}) + b_h(h_c - h),$$
$$\dot{V}_a = b_{V_a}(V_{a,c} - V_a).$$

Heading from wind triangle (for $\gamma_a = 0$):

$$\psi = \chi - \sin^{-1}\left(\frac{-w_n\sin\chi + w_e\cos\chi}{V_a}\right).$$

Option 2: roll as direct input

$$\dot{p}_n = V_a\cos\psi + w_n, \quad \dot{p}_e = V_a\sin\psi + w_e, \quad \dot{\psi} = \frac{g}{V_a}\tan\phi,$$
$$\ddot{h} = b_{\dot{h}}(\dot{h}_c - \dot{h}) + b_h(h_c - h), \quad \dot{V}_a = b_{V_a}(V_{a,c} - V_a), \quad \dot{\phi} = b_{\phi}(\phi_c - \phi).$$

Option 3: load-factor command

$$\dot{p}_n = V_a\cos\psi\cos\gamma_a + w_n, \quad \dot{p}_e = V_a\sin\psi\cos\gamma_a + w_e, \quad \dot{h} = V_a\sin\gamma_a - w_d,$$
$$\dot{\psi} = \frac{g}{V_a}\tan\phi, \quad \dot{\gamma} = \frac{g}{V_g}(n_{lf}\cos\phi - \cos\gamma),$$
$$\dot{n}_{lf} = b_n(n_{lf,c} - n_{lf}), \quad \dot{V}_a = b_{V_a}(V_{a,c} - V_a), \quad \dot{\phi} = b_{\phi}(\phi_c - \phi).$$

1.7 Dynamic point-mass guidance model

When lift/drag/thrust are retained explicitly:

$$\dot{p}_n = V_g \cos\chi \cos\gamma, \quad \dot{p}_e = V_g \sin\chi \cos\gamma, \quad \dot{h} = V_g \sin\gamma,$$

$$\dot{V}_g = \frac{F_{thrust}}{m} - \frac{F_{drag}}{m} - g\sin\gamma,$$

$$\dot{\chi} = \frac{F_{lift}\sin\phi\cos(\chi-\psi)}{mV_g\cos\gamma}, \quad \dot{\gamma} = \frac{F_{lift}}{mV_g}\cos\phi - \frac{g}{V_g}\cos\gamma.$$

Aerodynamic closure:

$$L = \frac{1}{2}\rho V_a^2 S C_L, \quad D = \frac{1}{2}\rho V_a^2 S C_D, \quad C_D = C_{D0} + K C_L^2.$$

1.8 Trajectory tracking vs path following

- **Trajectory tracking**: follow a time-parameterized reference.
- **Path following**: converge to geometric path, without prescribed timing.

For small UAVs in wind, path following is usually more robust and less control-intensive.

1.9 Straight-line path geometry

Desired line:

$$\mathcal{P}_{line}(r,q) = \{x \in \mathbb{R}^3 : \ x = r + \lambda q, \ \lambda \in \mathbb{R}\},$$

where r is path origin and q is path unit direction. Path-course angle:

$$\chi_q = \tan^{-1}\left(\frac{q_e}{q_n}\right).$$

Path-frame error:

$$e_p^P = \begin{bmatrix} e_{px} \\ e_{py} \\ e_{pz} \end{bmatrix} = R_i^P(p^i - r^i), \quad R_i^P = \begin{bmatrix} \cos\chi_q & \sin\chi_q & 0 \\ -\sin\chi_q & \cos\chi_q & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Cross-track dynamics:

$$\dot{e}_{py} = V_g \sin(\chi - \chi_q)\cos\gamma.$$

1.10 Longitudinal straight-line tracking

Desired altitude from projection geometry:

$$h_d = -r_d + \sqrt{s_n^2 + s_e^2}\left(\frac{q_d}{\sqrt{q_n^2 + q_e^2}}\right).$$

Altitude loop:

$$\ddot{h} = b_{\dot{h}}(\dot{h}_c - \dot{h}) + b_h(h_c - h).$$

If $h_c = h_d$ and loop is well tuned, steady-state altitude error to constant/ramp-type commands tends to zero.

1.11 Lateral straight-line vector-field strategy

Construct desired relative course:

$$\chi_d(e_{py}) = -\chi_\infty \frac{2}{\pi} \tan^{-1}(k_{path} e_{py}), \quad \chi_\infty \in (0, \pi/2],\ k_{path} > 0.$$

Then commanded course:

$$\chi_c = \chi_q + \chi_d(e_{py}).$$

Rule-of-thumb:

$$k_{path} \approx \frac{1}{R_{min}},$$

with R_{min} as minimum turning radius.

2 Lecture 14: Orbit Following, Carrot-Chasing, and NLGL

2.1 Lyapunov proof for straight-line lateral guidance

Use:

$$V(e_{py}) = \frac{1}{2}e_{py}^2, \qquad \chi_c = \chi_q - \chi_\infty \frac{2}{\pi}\tan^{-1}(k_{path}e_{py}).$$

Assuming inner loop makes $\chi \approx \chi_c$:

$$\dot{V} = e_{py}\dot{e}_{py} = V_g e_{py}\sin(\chi - \chi_q)\cos\gamma = -V_g e_{py}\sin\left(\chi_\infty \frac{2}{\pi}\tan^{-1}(k_{path}e_{py})\right)\cos\gamma < 0.$$

Hence $e_{py} \to 0$ asymptotically.

2.2 Orbit path representation and kinematics

Orbit:

$$\mathcal{P}_{orbit}(c, \rho, \lambda) = \{r \in \mathbb{R}^3 : \ r = c + \lambda\rho(\cos\varphi, \sin\varphi, 0)^T, \ \varphi \in [0, 2\pi)\}.$$

Here:

- $c =$ center, $\rho =$ radius,
- $\lambda = +1$ clockwise, $\lambda = -1$ counterclockwise,
- $d = \sqrt{(p_n - c_n)^2 + (p_e - c_e)^2}$.

Polar kinematics:

$$\dot{d} = V_g\cos(\chi - \varphi), \qquad \dot{\varphi} = \frac{V_g}{d}\sin(\chi - \varphi).$$

Desired tangent course:

$$\chi_o = \varphi + \lambda\frac{\pi}{2}.$$

2.3 Orbit vector-field command and stability

Course field:

$$\chi_d(d - \rho, \lambda) = \chi_o + \lambda\tan^{-1}\left(k_{orbit}\frac{d - \rho}{\rho}\right), \quad k_{orbit} > 0.$$

Lyapunov candidate:

$$V = \frac{1}{2}(d - \rho)^2.$$

With $\chi = \chi_d$:

$$\dot{V} = (d - \rho)V_g\cos(\chi_d - \varphi) = -V_g(d - \rho)\sin\left(\tan^{-1}\left(k_{orbit}\frac{d - \rho}{\rho}\right)\right) < 0, \quad d \neq \rho.$$

Therefore $d \to \rho$ asymptotically.

2.4 Practical angle wrapping issue

To avoid command jumps near $\pm\pi$, choose integer wrapping index m such that angle differences stay in $[-\pi, \pi]$. For orbit phase:

$$\varphi = \tan^{-1}\left(\frac{p_e - c_e}{p_n - c_n}\right) + 2\pi m, \quad m \in \mathbb{Z}.$$

2.5 Classification of path-following methods

- Geometric methods: pure pursuit, LOS, NLGL.
- Dubins-curve-based methods.
- Vector-field-based methods.

Many methods rely on a *virtual target point (VTP)* ahead on path.

2.6 Carrot-chasing algorithm

Straight-line mode:

- project UAV position on line segment,
- place VTP at look-ahead distance δ along line,
- command heading toward VTP.

Loiter mode:

- for desired circle of radius r, set VTP at angular offset λ ahead on orbit,
- command heading toward VTP.

2.7 Nonlinear Guidance Law (NLGL) / L1 Logic

Look-ahead point at distance L_1, angle η between velocity vector and line to look-ahead point:

$$L_1 = 2R\sin\eta \;\Rightarrow\; R = \frac{L_1}{2\sin\eta}.$$

Lateral acceleration command:

$$a_{s,cmd} = \frac{V^2}{R} = \frac{2V^2}{L_1}\sin\eta.$$

Small-angle perturbation about straight-line gives PD-like form:

$$\eta \approx \frac{d - d^*_{ref.pt}}{L_1} + \frac{\dot{d}}{V},$$

$$a_{s,cmd} \approx \frac{2V}{L_1}\dot{d} + \frac{2V^2}{L_1^2}(d - d^*_{ref.pt}).$$

Using $\ddot{d} = -a_{s,cmd}$:

$$\ddot{d} + \frac{2V}{L_1}\dot{d} + \frac{2V^2}{L_1^2}d = \frac{2V^2}{L_1^2}d^*_{ref.pt}.$$

Transfer function:

$$\frac{d(s)}{d^*_{ref.pt}(s)} = \frac{2(V/L_1)^2}{s^2 + (2V/L_1)s + 2(V/L_1)^2}.$$

Hence

$$\zeta = 0.707, \qquad \omega_n = \frac{\sqrt{2}V}{L_1}.$$

2.8 Look-ahead distance selection insight

- Small L_1 gives high gain, faster response, potentially aggressive control.
- For sinusoidal path with wavelength L_p, practical recommendation: $L_1 \lesssim L_p/4.4$ for good phase recovery near bandwidth.

2.9 PD/PID vs NLGL in wind

Observed in lecture comparisons:

- fixed-gain PD/PID on cross-track error can show larger deviations in strong wind and varying ground speed;
- NLGL naturally uses ground speed in acceleration command, improving performance in upwind/downwind sectors.

3 Lecture 15: Circular NLGL Analysis, Path Manager, Fillets, Dubins Basics

3.1 Circular path with nonlinear guidance assumptions

- Inner-loop control is sufficiently fast.
- All angular quantities are signed consistently.
- Look-ahead distance L_1 is constant.

Key geometry:

$$V_L = V\cos\eta, \qquad V_1 = V\sin\eta, \qquad V_T = \frac{V\cos\eta}{\cos\beta}, \qquad V_2 = V\cos\eta\tan\beta.$$

3.2 Dynamics of Eta and Beta

$$\dot{\eta} = -\frac{V}{L_1}\left(\sin\eta + \cos\eta\tan\beta\right),$$
$$\dot{\beta} = \frac{V}{L_1}\left(\sin\eta - \cos\eta\tan\beta\right) - \frac{V\cos\eta}{R\cos\beta}.$$

Stationary point from $\dot{\eta} = \dot{\beta} = 0$:

$$\eta_0 = -\beta_0, \qquad \eta_0 = \sin^{-1}\left(\frac{L_1}{2R}\right), \qquad L_1 \le 2R.$$

3.3 Linearization near circular equilibrium

Perturbations:

$$\eta = \eta_0 + \Delta\eta, \qquad \beta = \beta_0 + \Delta\beta.$$

Linearized second-order behavior yields:

$$\omega_n = \frac{\sqrt{2}V}{L_1}, \qquad \zeta = \frac{1}{\sqrt{2}}\sqrt{1 - \left(\frac{L_1}{2R}\right)^2}.$$

Compared to straight-line case, ω_n stays same while ζ depends on L_1/R.

3.4 Waypoint path manager motivation

A waypoint list:

$$\mathcal{W} = \{w_1, w_2, \ldots, w_N\}, \qquad w_i \in \mathbb{R}^3.$$

Simple "ball around waypoint" switching can fail under wind and short segments.

3.5 Half-plane switching

Half-plane:

$$\mathcal{H}(r, n) = \{p \in \mathbb{R}^3 : (p - r)^T n \ge 0\}.$$

Direction for segment $w_i \to w_{i+1}$:

$$q_i = \frac{w_{i+1} - w_i}{\|w_{i+1} - w_i\|}.$$

Switch at waypoint w_i using:

$$n_i = \frac{q_{i-1} + q_i}{\|q_{i-1} + q_i\|},$$

and transition once UAV enters $\mathcal{H}(w_i, n_i)$.

3.6 Fillet maneuver for smooth line transitions

Turn angle:

$$\varrho = \cos^{-1}(-q_{i-1}^T q_i).$$

For fillet radius R:

$$c = w_i - \frac{R}{\sin(\varrho/2)} \frac{q_{i-1} - q_i}{\|q_{i-1} - q_i\|},$$

$$r_1 = w_i - \frac{R}{\tan(\varrho/2)} q_{i-1}, \qquad r_2 = w_i + \frac{R}{\tan(\varrho/2)} q_i.$$

Procedure:

1. follow line $w_{i-1} \to w_i$ to half-plane at r_1,
2. follow circular arc (fillet) of radius R about center c,
3. switch to line $w_i \to w_{i+1}$ at half-plane through r_2.

3.7 Path-length correction with fillets

Nominal polyline length:

$$|W| = \sum_{i=2}^{N} \|w_i - w_{i-1}\|.$$

Fillet-corrected length:

$$|W|_F = |W| + \sum_{i=2}^{N} \left[R(\pi - \varrho_i) - \frac{2R}{\tan(\varrho_i/2)} \right].$$

This matters in cooperative timing and arrival synchronization.

3.8 Dubins vehicle and path decomposition

Bounded-curvature kinematics:

$$\dot{p}_n = V \cos\vartheta, \qquad \dot{p}_e = V \sin\vartheta, \qquad \dot{\vartheta} = u, \quad u \in [-\bar{u}, \bar{u}].$$

Minimum-turn radius:

$$R = \frac{V}{\bar{u}}.$$

Dubins path between (p_s, χ_s) and (p_e, χ_e) consists of two arcs and one straight segment (e.g., RSR, RSL, LSR, LSL).

3.9 Circle centers and angle operator

For pose (p, χ):

$$c_r = p + R(\cos(\chi + \pi/2), \sin(\chi + \pi/2), 0)^T,$$
$$c_l = p + R(\cos(\chi - \pi/2), \sin(\chi - \pi/2), 0)^T.$$

Use wrapped-angle operator:

$$\langle \phi \rangle = \phi \bmod 2\pi.$$

CW and CCW arc angle measures are defined with this wrapping.

3.10 Length formulas introduced in Lecture 15

RSR:

$$L_{RSR} = \|c_{rs} - c_{re}\| + d_1 + d_2,$$

with

$$d_1 = R\langle 2\pi + \langle \vartheta - \pi/2 \rangle - \langle \chi_s - \pi/2 \rangle \rangle,$$
$$d_2 = R\langle 2\pi + \langle \chi_e - \pi/2 \rangle - \langle \vartheta - \pi/2 \rangle \rangle.$$

RSL: Let $l = \|c_{le} - c_{rs}\|$, and

$$\vartheta_2 = \vartheta - \left(\frac{\pi}{2} - \sin^{-1} \frac{2R}{l} \right).$$

Then

$$L_{RSL} = \sqrt{l^2 - 4R^2} + d_3 + d_4,$$
$$d_3 = R\langle 2\pi + \langle \vartheta_2 \rangle - \langle \chi_s - \pi/2 \rangle \rangle,$$
$$d_4 = R\langle 2\pi + \langle \vartheta_2 + \pi \rangle - \langle \chi_e + \pi/2 \rangle \rangle.$$

4 Lecture 16: Dubins Completion and UAV Path Planning Algorithms

4.1 Remaining Dubins cases

LSR:

$$L_{LSR} = \sqrt{l^2 - 4R^2} + d_5 + d_6,$$
$$d_5 = R\langle 2\pi + \langle \chi_s + \pi/2\rangle - \langle \vartheta + \vartheta_2\rangle\rangle,$$
$$d_6 = R\langle 2\pi + \langle \chi_e - \pi/2\rangle - \langle \vartheta + \vartheta_2 - \pi\rangle\rangle.$$

LSL:

$$L_{LSL} = \|c_{ls} - c_{le}\| + d_7 + d_8,$$
$$d_7 = R\langle 2\pi + \langle \chi_s + \pi/2\rangle - \langle \vartheta + \pi/2\rangle\rangle,$$
$$d_8 = R\langle 2\pi + \langle \vartheta + \pi/2\rangle - \langle \chi_e + \pi/2\rangle\rangle.$$

Choose minimum among feasible RSR/RSL/LSR/LSL.

4.2 Path planning classes

- **Deliberative planning**: uses global map/model; explicit path computation; good for structured known environments.
- **Reactive planning**: uses local sensing and behavior rules; better for uncertain dynamic environments.

For many fixed-wing UAV missions, efficient deliberative planning with simplified motion/wind models is preferred.

4.3 Point-to-point planning with Voronoi graph

Given obstacle point set Q, Voronoi partition creates cells and edges equidistant from obstacles, producing safer corridors. Graph:

$$G = (V, E), \quad V^+ = V \cup \{p_s, p_e\}.$$

Start/end nodes are connected to nearest graph nodes.

4.4 Edge cost with safety-distance penalty

For edge (v_1, v_2) and obstacle point p, parameterize segment:

$$w(\sigma) = (1 - \sigma)v_1 + \sigma v_2, \quad \sigma \in [0, 1].$$

Distance minimization gives

$$D = \min_{\sigma \in [0,1]} \|p - w(\sigma)\|.$$

Unconstrained optimizer:

$$\sigma^* = -\frac{(p - v_1)^T (v_1 - v_2)}{\|v_1 - v_2\|^2}.$$

Use clipped distance $D'(v_1, v_2, p)$:

- $D(\sigma^*)$ if $\sigma^* \in [0, 1]$,
- $\|p - v_1\|$ if $\sigma^* < 0$,
- $\|p - v_2\|$ if $\sigma^* > 1$.

Distance from edge to obstacle set:

$$D(v_1, v_2, Q) = \min_{p \in Q} D'(v_1, v_2, p).$$

Edge cost:

$$J(v_1, v_2) = k_1 \|v_1 - v_2\| + \frac{k_2}{D(v_1, v_2, Q)}, \quad k_1, k_2 > 0.$$

4.5 Dijkstra search procedure

1. Initialize tentative distance 0 at start node, ∞ for others.
2. Repeatedly select unvisited node with minimum tentative distance.
3. Relax all unvisited neighbors.
4. Mark node visited.
5. Stop when destination visited or remaining tentative distances are ∞.

4.6 RRT for point-to-point planning

Rapidly Exploring Random Tree:

- randomly sample a configuration in workspace,
- find nearest node in existing tree,
- extend by fixed step D toward sample,
- add segment only if collision-free,
- continue until close enough to goal.

RRT extends naturally to nonlinear and higher-dimensional models, including 3D with climb/descent constraints.

4.7 Coverage path planning concept

Goal is area coverage (search), not just start-to-goal transit.

- Maintain terrain map (collision check) and return map (search benefit).
- Update return value after visit:

$$\Upsilon_i[k] = \Upsilon_i[k-1] - c, \quad c > 0.$$

- Build finite-depth look-ahead trees using motion primitives: step length L and heading changes $\{0, \pm\nu\}$ (example: $\nu = \pi/6$).

5 Lecture 17: Vision-Guided Navigation, Geolocation, and Time-to-Collision

5.1 Role of onboard vision

EO/IR cameras support:

- surveillance and situational awareness,
- guidance/control by using image features as measurements.

5.2 Frames and transformation chain

- Body frame: F_b
- Gimbal frame: F_g
- Camera frame: F_c

Body-to-gimbal-1 (azimuth rotation α_{az}):

$$R_b^{g1}(\alpha_{az}) = \begin{bmatrix} \cos\alpha_{az} & \sin\alpha_{az} & 0 \\ -\sin\alpha_{az} & \cos\alpha_{az} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Gimbal-1 to gimbal (elevation α_{el}):

$$R_{g1}^{g}(\alpha_{el}) = \begin{bmatrix} \cos\alpha_{el} & 0 & -\sin\alpha_{el} \\ 0 & 1 & 0 \\ \sin\alpha_{el} & 0 & \cos\alpha_{el} \end{bmatrix}.$$

Hence

$$R_b^g = R_{g1}^g(\alpha_{el}) R_b^{g1}(\alpha_{az}).$$

Camera alignment from gimbal basis:

$$i_c = j_g, \quad j_c = k_g, \quad k_c = i_g \Rightarrow R_g^c = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}.$$

5.3 Camera projection model

For image width M pixels and field of view ν:

$$f = \frac{M}{2\tan(\nu/2)}.$$

For pixel location (ϵ_x, ϵ_y):

$$F = \sqrt{f^2 + \epsilon_x^2 + \epsilon_y^2}, \qquad l_c = \frac{L}{F}\begin{bmatrix} \epsilon_x \\ \epsilon_y \\ f \end{bmatrix}, \qquad \check{l}_c = \frac{1}{F}\begin{bmatrix} \epsilon_x \\ \epsilon_y \\ f \end{bmatrix}.$$

Monocular camera gives direction $\check{l}_c$, not absolute distance L.

5.4 Gimbal pointing control

Pan-tilt kinematics:

$$\dot{\alpha}_{az} = u_{az}, \qquad \dot{\alpha}_{el} = u_{el}.$$

From desired LOS direction in body frame $\check{l}_d^b = [\check{l}_{xd}^b, \check{l}_{yd}^b, \check{l}_{zd}^b]^T$:

$$\alpha_{az}^c = \tan^{-1}\left(\frac{\check{l}_{yd}^b}{\check{l}_{xd}^b}\right), \qquad \alpha_{el}^c = -\sin^{-1}(\check{l}_{zd}^b).$$

Simple proportional servo commands:

$$u_{az} = k_{az}(\alpha_{az}^c - \alpha_{az}), \qquad u_{el} = k_{el}(\alpha_{el}^c - \alpha_{el}).$$

5.5 Geolocation setup

Object position:

$$p_{obj}^i = p_{MAV}^i + R_b^i R_g^b R_c^g \, l_c = p_{MAV}^i + L \, R_b^i R_g^b R_c^g \, \check{l}_c.$$

Unknown L causes scale ambiguity.

5.6 Flat-earth geolocation estimate

If camera-ground intersection uses known vehicle height h and flat-earth assumption:

$$L = \frac{h}{\cos\phi}, \qquad \cos\phi = k_i^T \left(R_b^i R_g^b R_c^g \check{l}_c\right).$$

Then:

$$p_{obj}^i = \begin{bmatrix} p_n \\ p_e \\ p_d \end{bmatrix} + h \, \frac{R_b^i R_g^b R_c^g \check{l}_c}{k_i^T \left(R_b^i R_g^b R_c^g \check{l}_c\right)}.$$

5.7 EKF formulation for geolocation

State:

$$\hat{x} = \begin{bmatrix} \hat{p}_{obj}^i \\ \hat{L} \end{bmatrix}.$$

Prediction equations (stationary object assumption):

$$\dot{\hat{p}}_{obj}^i = 0, \qquad \dot{\hat{L}} = -\frac{(\hat{p}_{obj}^i - \hat{p}_{MAV}^i)^T \dot{\hat{p}}_{MAV}^i}{\hat{L}}.$$

Measurement relation:

$$p_{MAV}^i = p_{obj}^i - L R_b^i R_g^b R_c^g \check{l}_c.$$

Jacobian terms follow from these dynamics and measurement model.

5.8 Target pixel filtering and velocity estimation

Raw pixel vector $\bar{\epsilon} = [\bar{\epsilon}_x, \bar{\epsilon}_y]^T$ is noisy. Use low-pass and filtered derivative:

$$\epsilon(s) = \frac{1}{\tau s + 1}\bar{\epsilon}, \qquad \dot{\epsilon}(s) = \frac{s}{\tau s + 1}\bar{\epsilon}.$$

Using Tustin discretization:

$$\epsilon[n] = \frac{2\tau - T_s}{2\tau + T_s}\epsilon[n-1] + \frac{T_s}{2\tau + T_s}\left(\bar{\epsilon}[n] + \bar{\epsilon}[n-1]\right),$$

$$\dot{\epsilon}[n] = \frac{2\tau - T_s}{2\tau + T_s}\dot{\epsilon}[n-1] + \frac{2}{2\tau + T_s}\left(\bar{\epsilon}[n] - \bar{\epsilon}[n-1]\right).$$

5.9 Removing apparent motion due to platform/gimbal rotation

Rate transport:

$$\frac{d\check{l}_c}{dt_i} = \frac{d\check{l}_c}{dt_c} + \omega^c_{c/i} \times \check{l}_c.$$

Normalized LOS derivative from pixel rates:

$$\frac{d\check{l}_c}{dt_c} = Z(\epsilon)\dot{\epsilon},$$

where

$$Z(\epsilon) = \frac{1}{F^3}\begin{bmatrix} \epsilon_y^2 + f^2 & -\epsilon_x\epsilon_y \\ -\epsilon_x\epsilon_y & \epsilon_x^2 + f^2 \\ -\epsilon_x f & -\epsilon_y f \end{bmatrix}.$$

Subtracting rotation-induced term isolates translationally useful motion.

5.10 Time-to-collision (TTC)

Definition:

$$t_c = \frac{L}{\dot{L}}.$$

With monocular camera only, TTC is ambiguous unless additional information is used.

If object physical size S_{obj} and image size ϵ_s are available:

$$\frac{S_{obj}}{L} = \frac{\epsilon_s}{F}.$$

Differentiation gives:

$$\frac{\dot{L}}{L} = \frac{\epsilon_x\dot{\epsilon}_x + \epsilon_y\dot{\epsilon}_y}{F} - \frac{\dot{\epsilon}_s}{\epsilon_s},$$

hence

$$t_c = \frac{1}{\dfrac{\epsilon_x\dot{\epsilon}_x + \epsilon_y\dot{\epsilon}_y}{F} - \dfrac{\dot{\epsilon}_s}{\epsilon_s}}.$$

6 Consolidated Revision Checklist (Lectures 13–17)

1. Derive and interpret coordinated-turn and accelerated-climb equations.
2. Explain why path following is preferred over trajectory tracking for small UAVs in wind.
3. Reproduce straight-line and orbit vector-field commands and Lyapunov proofs.
4. Derive NLGL linearized transfer function and identify ζ, ω_n.
5. Compute half-plane switches and fillet geometry from waypoint triplets.
6. Evaluate Dubins path length for all four cases and pick minimum feasible route.
7. Build Voronoi graph edge costs and describe Dijkstra steps.
8. Explain RRT growth, collision checking, and stopping criteria.
9. Write complete frame chain for vision geolocation and identify source of scale ambiguity.
10. Derive TTC from pixel-size change and discuss assumptions.

Primary Reference

Randal Beard and Timothy W. McLain, *Small Unmanned Aircraft: Theory and Practice*, Princeton University Press, 2012.